Rising Waters: A Machine Learning Approach to Flood Prediction

Project Description:

Rising Waters is a machine learning-powered flood prediction system that predicts flood risk using historical weather and rainfall data. The project uses Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost algorithms and integrates the best model into a Flask web application for real-time flood prediction.

Floods are among the most devastating natural disasters, claiming thousands of lives and displacing millions every year. Despite their recurring nature, the lack of timely and accurate early-warning systems continues to amplify their destructive impact. Conventional forecasting methods often fall short in predicting floods at the right time, leaving authorities and communities with insufficient opportunity to respond.

Scenarios

Scenario 1:. Early Flood Warning and Evacuation Planning

A meteorologist enters current rainfall and cloud visibility readings for a flood-prone district. The model analyses the inputs and predicts a high probability of flooding, allowing authorities to issue evacuation advisories several hours in advance.

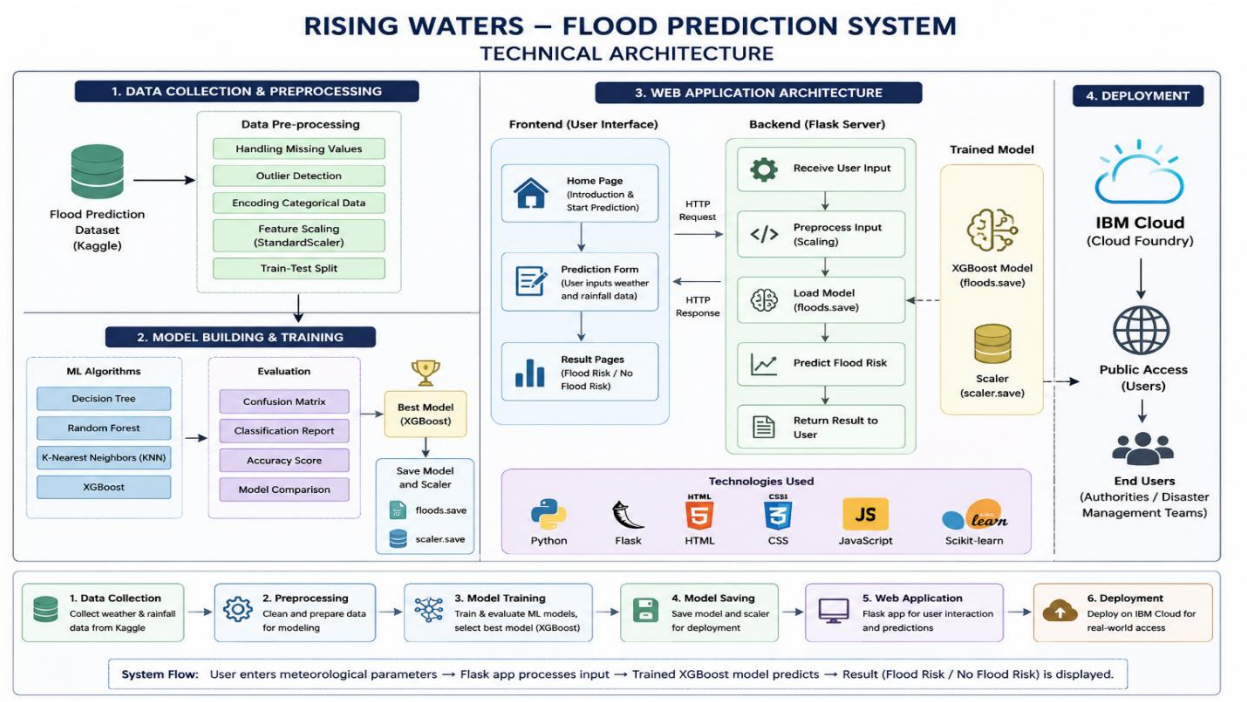
Scenario 2: Disaster Response and Resource Allocation

A disaster relief coordinator uses the web application during monsoon season to monitor multiple regions simultaneously. By entering regional weather data for each area, the system provides instant flood risk classifications, helping prioritise resource deployment.

Scenario 3: Model Validation and Performance Assessment

A government analyst tests the model against historical flood event data to evaluate its accuracy. The XGBoost model achieves 96.55% accuracy on test data, confirming the system's reliability for operational use.

Technical Architecture:



Description: The Rising Waters – Flood Prediction System uses machine learning to predict flood risk using historical weather and rainfall data.The system preprocesses meteorological data and trains multiple models, including Decision Tree, Random Forest, KNN, and XGBoost.The best-performing model, XGBoost, is integrated into a Flask web application for real-time predictions.Users can enter environmental parameters and instantly receive Flood Risk or No Flood Risk results.The application is designed for deployment on IBM Cloud to support disaster preparedness and early warning systems.

Pre-requisites:

- **Python Programming Knowledge:** Python Documentation
- **Machine Learning Fundamentals:** Scikit-learn Documentation
- **Data Analysis and Visualization:** Pandas and Matplotlib Documentation
- **Flask Framework Knowledge:** Flask Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **XGBoost Library Familiarity:** XGBoost Documentation
- **Version Control with Git:** Git Documentation
- **Development Environment Setup:** Anaconda Documentation and Jupyter Notebook Documentation
- **IBM Cloud Fundamentals:** IBM Cloud Documentation
- **Python Package Management:** Pip Documentation

Project Workflow:

Milestone 1: Environment Setup and System Architecture

- **Activity 1.1:** Set up the development environment using Python, Anaconda Navigator, and Jupyter Notebook.
- **Activity 1.2:** Install the required libraries such as NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn, Flask, and XGBoost.
- **Activity 1.3:** Define the architecture of the flood prediction system, including the frontend, backend, machine learning models, and deployment components.
- **Activity 1.4:** Configure the project structure and prepare the development environment for model building and web application development.

Milestone 2: Dataset Collection and Data Analysis

- **Activity 2.1:** Develop the core functionalities: Caption Generator, Hashtag Suggester, and Call-to-Action Generator.
- **Activity 2.2:** Implement the Flask backend to manage routing and user input processing, ensuring smooth API interactions.

Milestone 3: Data Pre-processing and Model Development

- **Activity 3.1:** Clean and preprocess the dataset by handling missing values, detecting outliers, and applying feature scaling techniques.
- **Activity 3.2:** Split the dataset into training and testing datasets and prepare features for model training.
- **Activity 3.3:** Train and evaluate multiple machine learning models, including Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost.
- **Activity 3.4:** Select the best-performing model based on evaluation metrics and save the trained model for deployment.

Milestone 4: Flask Web Application Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript to provide an intuitive flood prediction platform.
- **Activity 4.2:** Implement the Flask backend to accept user inputs, load the trained model, and generate real-time flood predictions.
- **Activity 4.3:** Create dynamic web pages, including the Home Page, Prediction Input Page, Flood Chance Result Page, and No Flood Chance Result Page.

Milestone 5: Deployment and Testing

- **Activity 5.1:** Prepare the application for local deployment by configuring the Flask server and installing all required dependencies.
- **Activity 5.2:** Test the web application using different weather inputs and validate prediction accuracy and functionality.
- **Activity 5.3:** Prepare the application for deployment on IBM Cloud to provide scalable and remote access to flood prediction services

Milestone 6: Conclusion

Milestone 1: Environment Setup and System Architecture

In this milestone, the development environment for the Rising Waters – Flood Prediction System was prepared and the overall system architecture was designed. The necessary software, libraries, and tools required for data analysis, machine learning model development, and web application implementation were installed and configured. The flood prediction dataset was studied to understand the meteorological parameters influencing flood occurrences. The system architecture was then defined to illustrate the interaction between the user interface, machine learning model, and prediction engine.

Activity 1.1: Install and Configure the Development Environment

Before starting the project development, the necessary tools and software need to be installed and configured.

Step 1 – Install Python
Download and install **Python 3.8 or above** from the official Python website:
https://www.python.org/downloads/

Step 2 – Install Anaconda Navigator
Download and install **Anaconda Navigator** to manage Python environments and Jupyter Notebook:
https://www.anaconda.com/products/distribution

Step 3 – Create a Virtual Environment
Create and activate a virtual environment to isolate project dependencies:
python -m venv myenv
myenv\Scripts\activate

Step 4 – Install Required Libraries
Install the required packages using pip:
pip install numpy pandas matplotlib seaborn scikit-learn flask xgboost joblib

Step 5 – Verify the Installation
Verify that all libraries are installed successfully:
pip list

Step 6 – Set Up the Project Structure
Create the project directory structure containing:

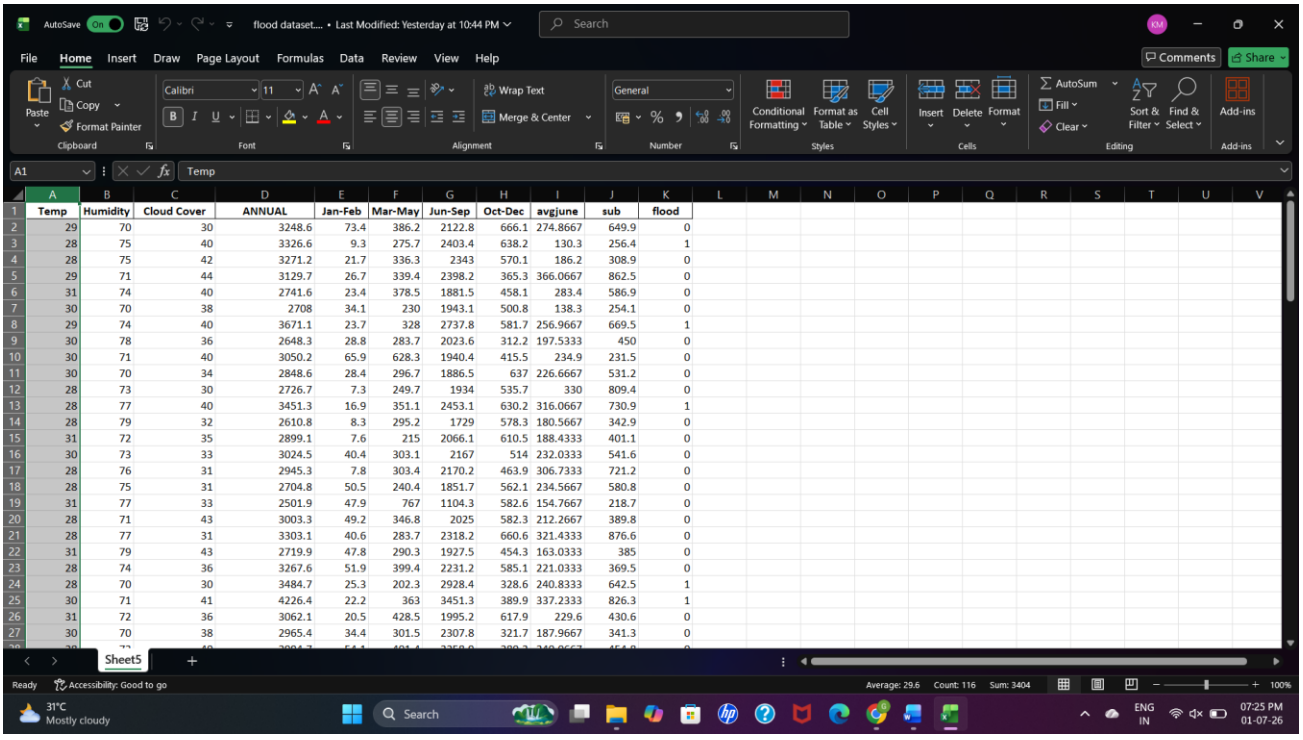
- templates/
- static/
- dataset/
- app.py
- requirements.txt
- floods.save
- scaler.save

Activity 1.2: Understand the Project Requirements and Dataset

The flood prediction dataset was collected from an open-source platform and analysed to understand the features influencing flood occurrences. The dataset contains meteorological parameters such as temperature, humidity, cloud cover, annual rainfall, and seasonal rainfall measurements.

The target variable of the dataset is Flood, which indicates whether a flood event is likely to occur**.**

- Study the impact of floods and the importance of early warning systems.
- Analyze the flood prediction dataset and identify the meteorological features used for prediction.
- Understand the target variable (**Flood / No Flood**).
- Identify suitable machine learning algorithms for classification.
- Determine the requirements for developing a web-based flood prediction system.



Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.8667	649.9	0
28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.3	256.4	1
28	75	42	3271.2	21.7	336.3	2343	570.1	186.2	308.9	0
29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.0667	862.5	0
31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.4	586.9	0
30	70	38	2708	34.1	230	1943.1	500.8	138.3	254.1	0
29	74	40	3671.1	23.7	328	2737.8	581.7	256.9667	669.5	1
30	78	36	2648.3	28.8	283.7	2023.6	312.2	197.5333	450	0
30	71	40	3050.2	65.9	628.3	1940.4	415.5	234.9	231.5	0
30	70	34	2848.6	28.4	296.7	1886.5	637	226.6667	531.2	0
28	73	30	2726.7	7.3	249.7	1934	535.7	330	809.4	0
28	77	40	3451.3	16.9	351.1	2453.1	630.2	316.0667	730.9	1
28	79	32	2610.8	8.3	295.2	1729	578.3	180.5667	342.9	0
31	72	35	2899.1	7.6	215	2066.1	610.5	188.4333	401.1	0
30	73	33	3024.5	40.4	303.1	2167	514	232.0333	541.6	0
28	76	31	2945.3	7.8	303.4	2170.2	463.9	306.7333	721.2	0
28	75	31	2704.8	50.5	240.4	1851.7	562.1	234.5667	580.8	0
31	77	33	2501.9	47.9	767	1104.3	582.6	154.7667	218.7	0
28	71	43	3003.3	49.2	346.8	2025	582.3	212.2667	389.8	0
28	77	31	3303.1	40.6	283.7	2318.2	660.6	321.4333	876.6	0
31	79	43	2719.9	47.8	290.3	1927.5	454.3	163.0333	385	0
28	74	36	3267.6	51.9	399.4	2231.2	585.1	221.0333	369.5	0
28	70	30	3484.7	25.3	202.3	2928.4	328.6	240.8333	642.5	1
30	71	41	4226.4	22.2	363	3451.3	389.9	337.2333	826.3	1
31	72	36	3062.1	20.5	428.5	1995.2	617.9	229.6	430.6	0
30	70	38	2965.4	34.4	301.5	2307.8	321.7	187.9667	341.3	0

Steps Performed

- Collected the dataset.
- Inspected the dataset features.
- Checked the data types and target variable.
- Identified the input and output variables.

Activity 1.3: Define the Architecture of the Application

Draft an Architectural Diagram

Create a visual representation of the system architecture, including:

- Dataset Collection and Preprocessing
- Machine Learning Model Development

- Flask Web Application
- IBM Cloud Deployment

Detail Frontend Functionality

Define how users interact with the application by:

- Accessing the Home Page
- Entering meteorological parameters
- Submitting the prediction request
- Viewing the flood prediction result

Outline Backend Responsibilities

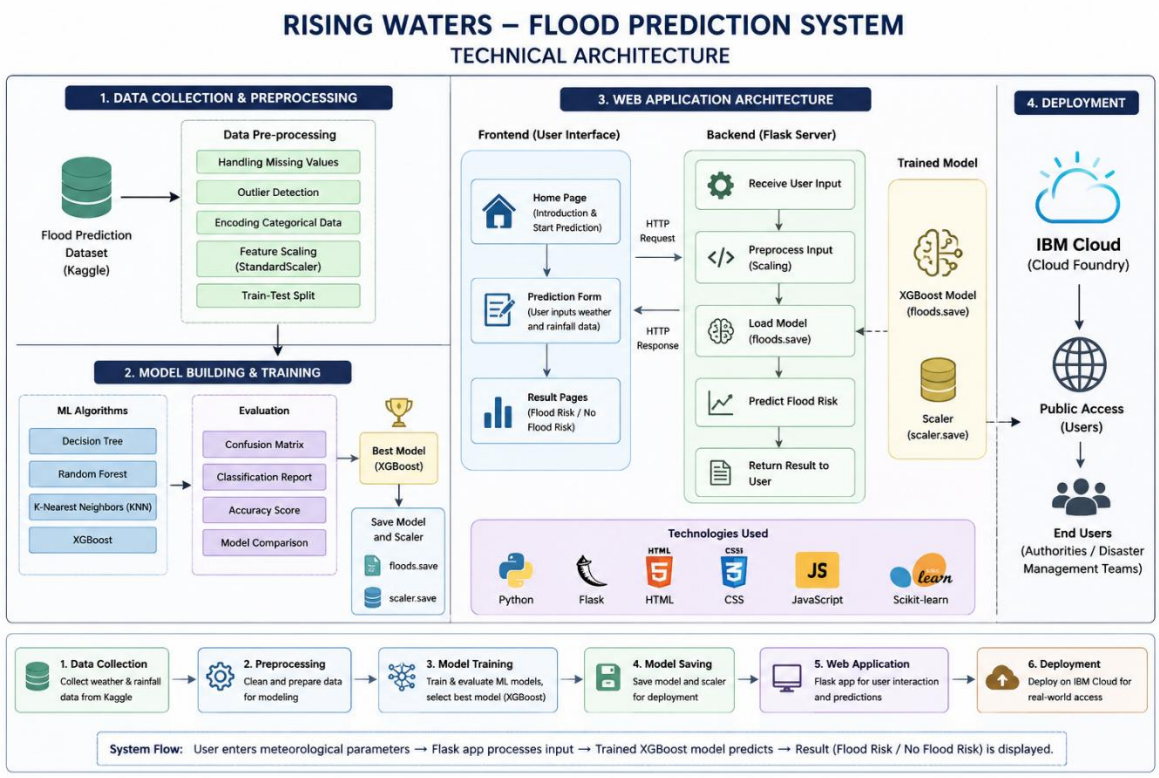
Specify how the Flask backend:

- Receives user input
- Preprocesses the data
- Loads the trained model
- Generates predictions
- Returns the prediction result to the user

Describe Model Integration

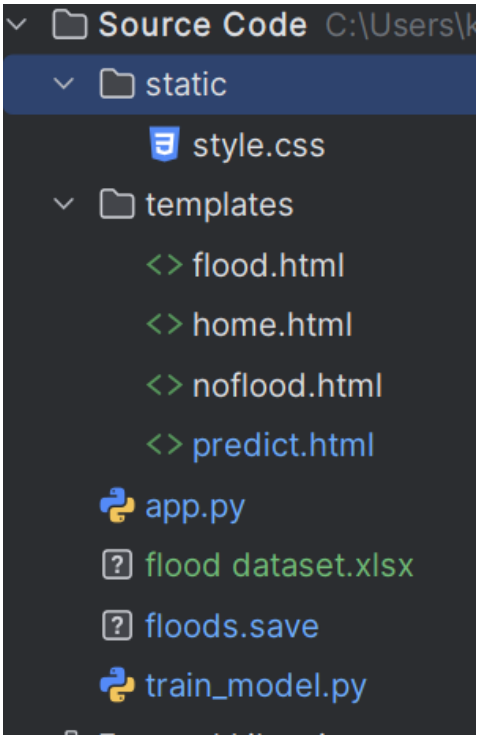
Define how the application:

- Loads the saved model (**floods.save**)
- Loads the scaler (**scaler.save**)
- Processes user inputs
- Predicts **Flood Risk** or **No Flood Risk**



Activity 1.4: Prepare the Development Environment for Model Building

- Install Jupyter Notebook and configure the Python environment.
- Import the required libraries for data analysis and machine learning.
- Configure the project folders and dataset location.
- Verify that Flask and the machine learning libraries are working correctly.
- Prepare the environment for model training and web application development.



Milestone 2: Dataset Collection and Data Analysis

In this milestone, the flood prediction dataset was collected and analyzed to understand the factors that influence flood occurrences. The dataset consists of historical weather and rainfall parameters such as temperature, humidity, cloud cover, annual rainfall, and seasonal rainfall measurements. Exploratory Data Analysis (EDA) techniques were applied to identify patterns, correlations, and trends within the dataset. The insights obtained from this analysis helped in understanding the relationship between different meteorological parameters and flood events and provided a strong foundation for data preprocessing and model development.

Activity 2.1: Collect the Flood Prediction Dataset

The dataset used for this project was collected from open-source platforms such as **Kaggle**. The dataset contains historical weather information and flood occurrence records, which are used to train and evaluate machine learning models.

Dataset Features

- Temperature (Temp)
- Humidity
- Cloud Cover
- Annual Rainfall (ANNUAL)
- Jan-Feb Rainfall
- Mar-May Rainfall
- Jun-Sep Rainfall
- Oct-Dec Rainfall
- Average June Rainfall (avgjune)
- Subdivision (sub)
- Flood (Target Variable)

Steps Performed

- Downloaded the flood prediction dataset.
- Loaded the dataset into Jupyter Notebook using Pandas.
- Examined the dataset structure and feature information.
- Identified the dependent and independent variables.

Activity 2.2: Perform Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to understand the characteristics of the dataset and identify patterns that could influence flood prediction.

Analysis Performed

- Univariate Analysis
- Multivariate Analysis
- Descriptive Statistics
- Distribution Plots
- Box Plots
- Heat Maps

The analysis helped identify:

- Data distribution patterns
- Relationships between features
- Presence of outliers
- Correlation between rainfall parameters and flood occurrences

Milestone 3: Data Pre-processing and Model Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the CaptionAI application. In this milestone, you'll set up each feature's functionality through Flask routes, establish reliable prompt construction and API interaction, and conduct unit testing to ensure each feature performs as expected.

Activity 3.1: Data Pre-processing and Preparation

Before training the machine learning models, the dataset was preprocessed to improve data quality and ensure accurate predictions.

Perform Data Cleaning

- Load the dataset using Pandas.
- Check for missing or inconsistent values.
- Remove duplicate records if present.
- Verify data types of all features.

Handle Outliers and Data Distribution

- Analyze feature distributions using box plots and histograms.
- Identify potential outliers in rainfall and weather parameters.
- Apply appropriate preprocessing techniques to prepare the data.

Define Input and Output Variables

- Independent Variables (X):
  - Temperature
  - Humidity
  - Cloud Cover
  - Annual Rainfall
  - Seasonal Rainfall Features
- Dependent Variable (Y):
  - Flood (0 – No Flood, 1 – Flood)

Apply Feature Scaling

- Normalize the numerical features using feature scaling techniques.
- Prepare the data for machine learning model training.

Split the Dataset

- Divide the dataset into training and testing datasets.
- Use an appropriate train-test split ratio for model evaluation.

Activity 3.2: Build and Train Machine Learning Models

Multiple machine learning classification algorithms were trained and compared to identify the most suitable model for flood prediction.

Models Implemented

- Decision Tree Classifier
- Random Forest Classifier
- K-Nearest Neighbours (KNN)
- XGBoost Classifier

Steps Performed

- Train each model using the training dataset.
- Generate predictions using the testing dataset.
- Compare model performance using evaluation metrics.

Activity 3.3: Model Evaluation and Performance Analysis

The trained models were evaluated using different performance metrics to determine their effectiveness in predicting flood events.

Evaluation Techniques

- Accuracy Score
- Confusion Matrix
- Classification Report
- Precision
- Recall
- F1-Score

The performance comparison showed that **XGBoost** achieved the highest prediction accuracy and outperformed the other classification algorithms.

Activity 3.4: Best Model Selection and Saving the Model

After evaluating all the models, **XGBoost** was selected as the best-performing model for the flood prediction system.

Steps Performed

- Compare the performance of all models.
- Select the model with the highest accuracy.
- Save the trained model using Joblib/Pickle.
- Save the scaler for preprocessing future user inputs.

Saved Files

- floods.save
- scaler.save

The saved model is later integrated into the Flask web application to provide real-time flood risk predictions.

Milestone 4: Flask Web Application Development

Milestone 4 focuses on developing a user-friendly and interactive web application for the Rising Waters – Flood Prediction System. This milestone involves designing an intuitive frontend interface using HTML, CSS, and JavaScript and integrating it with the Flask backend to provide real-time flood risk predictions. The web application is designed to enable users to enter meteorological parameters easily and obtain instant flood prediction results through a responsive and accessible interface.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

The user interface was developed using HTML to create a simple and easy-to-use web application for flood prediction.

The application consists of the following pages:

- **Home Page:** Introduces the project and provides an overview of the flood prediction system.
- **Prediction Input Page:** Allows users to enter weather and rainfall parameters required for prediction.
- **Flood Chance Result Page:** Displays the prediction result when a flood is likely to occur.
- **No Flood Chance Result Page:** Displays the prediction result when no flood risk is detected.

The HTML pages were designed using semantic elements to maintain a clear structure and improve readability and accessibility.

Design a Responsive Layout Using CSS

CSS was used to create an attractive and responsive interface that can be accessed on different devices, including desktops, tablets, and mobile phones.

The user interface includes:

- Well-organized input forms with clearly labeled fields.
- Responsive buttons and navigation components.
- Background images and styling to improve visual appearance.
- Proper alignment and spacing of page elements.
- Responsive design using CSS media queries.

The layout was designed to ensure that users can easily navigate through the application and obtain prediction results without difficulty.

Create User Interface Components

Several user interface components were created to improve usability and user experience.

Input Form Components

- Text boxes for entering weather and rainfall parameters.
- Input validation to ensure valid numerical values are entered.
- Predict button to submit user inputs.

Result Components

- Prediction message displaying **Flood Chance** or **No Flood Chance**.
- Separate result pages for positive and negative predictions.
- Navigation buttons to return to the prediction page.

Navigation Components

- Home button and page navigation links.
- Informative text describing the purpose of the application.

The user interface was designed to provide a simple and efficient flood prediction experience for meteorologists, disaster management authorities, and general users.

Activity 4.2: Integrating Frontend with Flask Backend

Handle User Input Submission

The prediction form was integrated with the Flask backend to process user inputs and generate flood predictions.

Steps Performed

- Accept user input through HTML forms.
- Validate the entered values.
- Send the data to the Flask application for processing.
- Preprocess the input data before prediction.

Generate Predictions

The Flask application loads the trained machine learning model and performs flood prediction based on the user-provided values.

Backend Processing Steps

- Receive input values from the prediction form.
- Apply the same preprocessing techniques used during model training.
- Load the saved machine learning model.
- Generate the prediction result.
- Return the result to the user interface.

Display Prediction Results

The prediction results are dynamically displayed to the user.

Result Display Features

- Display **Flood Chance** when the model predicts a possible flood event.
- Display **No Flood Chance** when the model predicts no flood risk.
- Present the result in a clear and user-friendly manner.
- Allow users to perform multiple predictions by returning to the input page.

Ensure Responsive User Experience

The application was tested on different screen sizes to ensure consistent performance and accessibility.

Features Implemented

- Responsive design for various devices.
- Easy navigation between pages.
- Fast prediction generation and result display.
- User-friendly interface with minimal complexity.

Outcome of Milestone 4

By completing this milestone, a fully functional web application was developed that integrates the machine learning model with a user-friendly interface. The application enables users to provide meteorological data and instantly receive flood risk predictions, thereby supporting early warning systems and disaster preparedness initiatives.

Milestone 5: Deployment and Testing

In this milestone, the focus is on preparing the **Rising Waters – Flood Prediction System** for deployment and verifying that the application functions correctly in a local environment. This involves configuring the Flask application, managing dependencies, running the application locally, and validating the prediction results using different weather and rainfall inputs. The system is also prepared for future deployment on IBM Cloud to provide scalable and remote access to flood prediction services.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment

Before running the application, the required dependencies were installed in an isolated Python environment.

Open a New Terminal

Open Command Prompt or Terminal and navigate to the project directory.

Create and Activate a Virtual Environment

```
python -m venv myenv
myenv\Scripts\activate
```

Install Required Dependencies

```
pip install -r requirements.txt
```

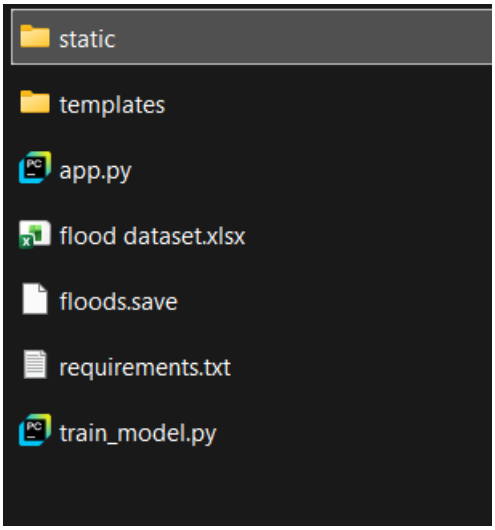
or

```
pip install numpy pandas scikit-learn flask xgboost joblib
```

Verify Installed Packages

```
pip list
```

The virtual environment ensures that all project dependencies are managed separately and avoids conflicts with other Python projects.



Project Folder Structure of the Rising Waters Application

```
PS C:\Users\keert\OneDrive\Desktop\Rising-Waters-Flood-Prediction> pip install -r requirements.txt
Defaulting to user installation because normal site-packages is not writeable

[notice] A new release of pip is available: 25.1.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
ERROR: Could not open requirements file: [Errno 2] No such file or directory: 'requirements.txt'
PS C:\Users\keert\OneDrive\Desktop\Rising-Waters-Flood-Prediction>
```

Terminal Showing Installed Dependencies and Required Package

Activity 5.2: Local Testing and Verification

Start the Flask Development Server

Run the application locally using:

python app.py

Once the server starts successfully, open a web browser and navigate to:

http://127.0.0.1:5000

```
"C:\Program Files\Python313\python.exe" "C:\Users\keert\OneDrive\Desktop\Rising-Waters-Flood-Prediction\5. Project Development Phase\Source Co
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 116-040-919
```

Flask Development Server Running Successfully

Verify Application Functionality

The following tests were performed:

- Enter different weather and rainfall parameters.
- Verify that the prediction page accepts valid inputs.
- Check whether the model correctly predicts **Flood Chance** or **No Flood Chance**.
- Test multiple input combinations to ensure consistent performance.
- Validate that the application displays results without errors.

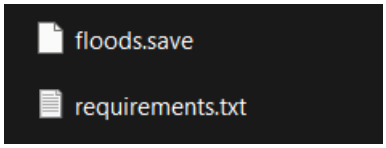
The successful execution of these tests confirms that the web application is functioning correctly

Activity 5.3: Deployment Preparation

The application was designed to support future deployment on **IBM Cloud**.

Deployment Preparation Steps

- Organize the project files and dependencies.
- Prepare the requirements.txt file.
- Verify that the Flask application runs successfully in the local environment.
- Ensure that the saved machine learning model and scaler are properly loaded by the application.
- Prepare the project structure for cloud deployment and scalability.



Saved Model Files and Project Dependencies

Future Enhancements

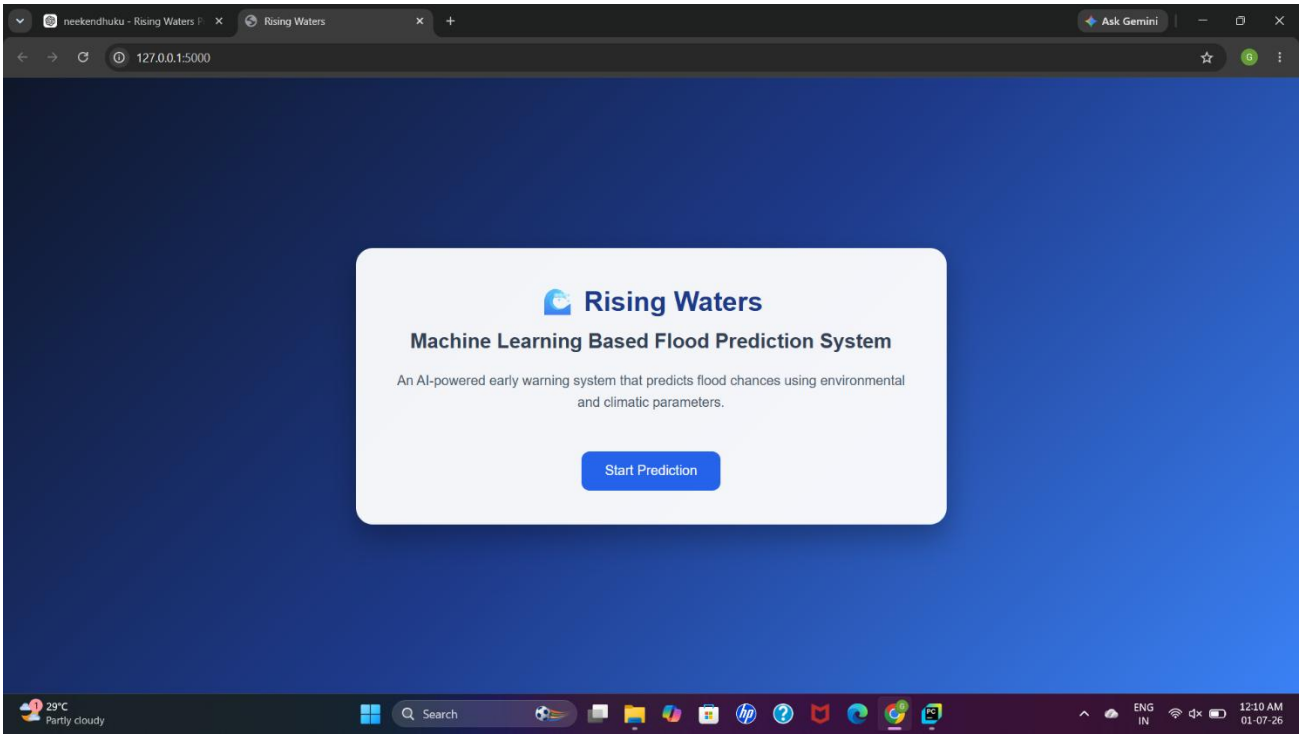
- Deploy the application on IBM Cloud.
- Integrate real-time weather APIs.
- Implement automated flood alert notifications.
- Enable remote access for disaster management authorities and government agencies.

Important Notes

- The current version of the **Rising Waters – Flood Prediction System** runs successfully in a local environment using Flask.
- The trained **XGBoost model** is integrated into the web application to provide real-time flood risk predictions.
- The application is designed for future deployment on **IBM Cloud** to improve accessibility and scalability.
- The system can be further enhanced by integrating **real-time weather APIs** and **automated flood alert notifications**.
- The prediction results depend on the quality and accuracy of the historical weather dataset used for model training.

Exploring the Website's Web Pages:

Home / Generator Page:



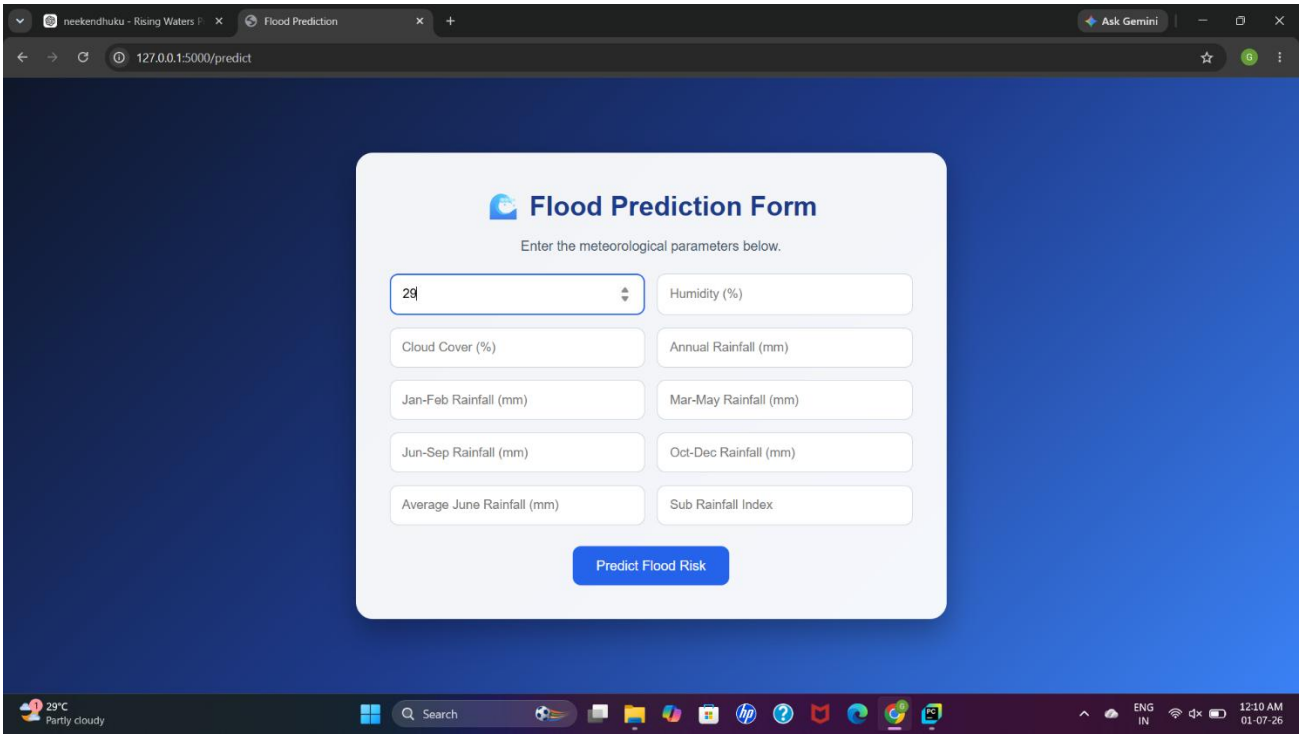
Description: The Home Page serves as the main entry point of the Rising Waters – Flood Prediction System and provides users with an overview of the application's purpose and functionality. The page is designed with a simple, responsive, and user-friendly interface to ensure easy accessibility for all users, including meteorologists, disaster management authorities, and government agencies.

The homepage introduces the importance of flood prediction and highlights how machine learning can be used to analyze historical weather and rainfall data to predict potential flood events. It contains a brief description of the system, its objectives, and the benefits of early flood prediction in disaster preparedness and resource management.

The page also provides navigation to the flood prediction module, where users can enter meteorological parameters such as rainfall and weather conditions to obtain real-time flood risk predictions. The clean layout and intuitive design make it easy for users to understand the application and access its features efficiently.

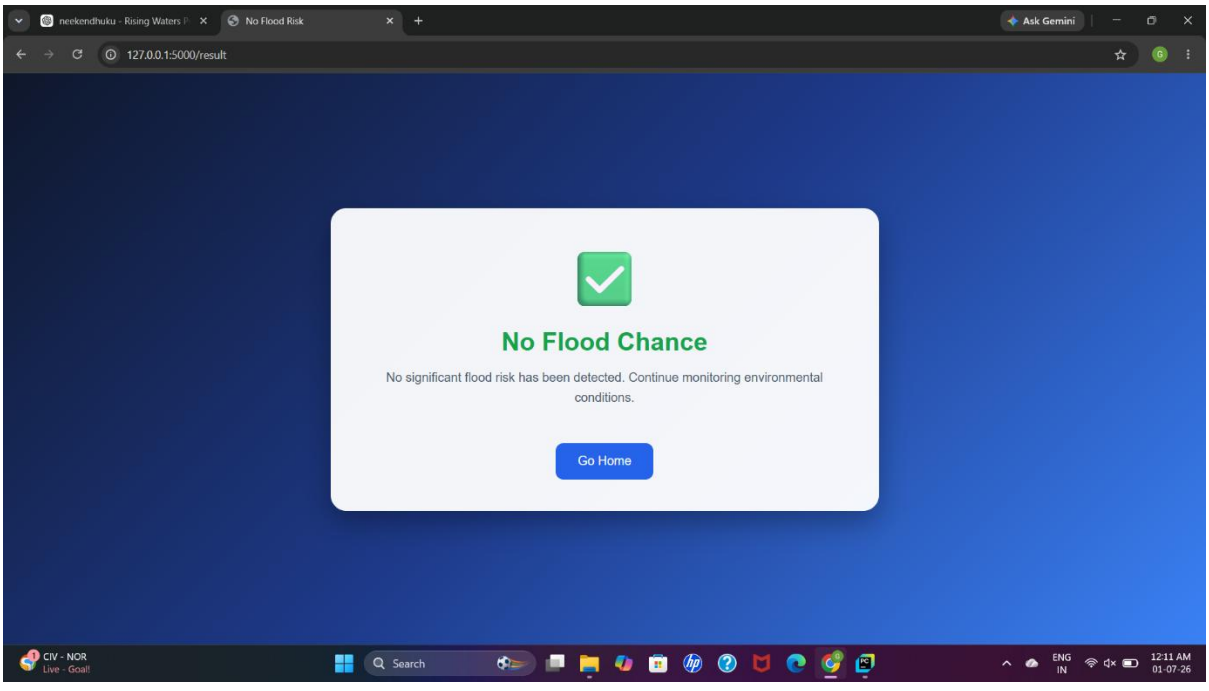
Overall, the Home Page acts as an informative gateway to the system, guiding users toward the prediction interface and promoting the importance of proactive flood management and early warning systems.

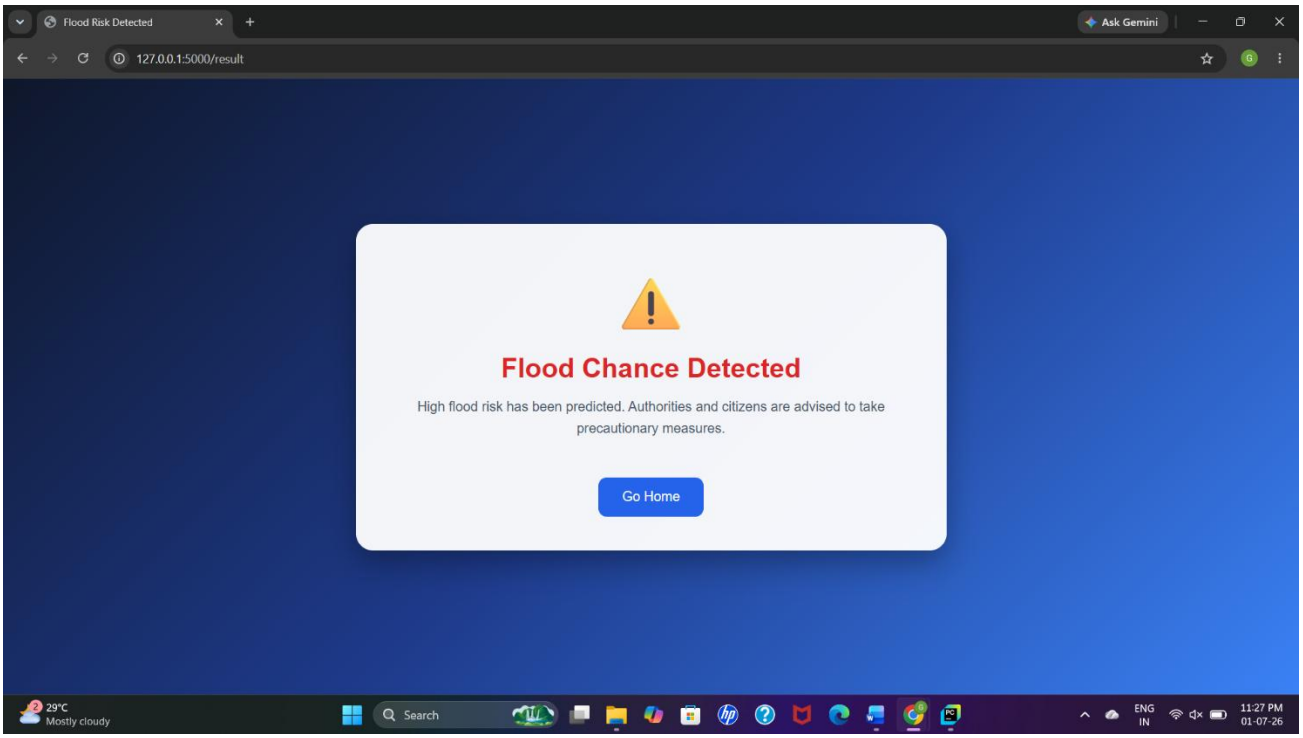
Input Section:



Description: The Input Section is the core component of the Rising Waters – Flood Prediction System, where users provide the meteorological parameters required for flood prediction.The Input Section plays a vital role in enabling real-time flood risk assessment by allowing users to quickly provide environmental data and receive instant predictions. Its user-friendly design ensures that the application can be effectively used by meteorologists, disaster management authorities, and other stakeholders involved in flood monitoring and early warning systems.

Results Section:





Description: The Result Section displays the flood prediction generated by the trained machine learning model based on the user-provided weather and rainfall parameters. After the user submits the input values, the application processes the data and presents the prediction result as either Flood Chance or No Flood Chance.

This section plays a crucial role in the system by transforming complex machine learning predictions into meaningful information that can be used by meteorologists, disaster management authorities, and government agencies for early warning, resource allocation, and evacuation planning. The simple and intuitive presentation of results ensures that users can interpret the predictions efficiently and take appropriate actions when necessary.

Conclusion:

Rising Waters: A Machine Learning Approach to Flood Prediction demonstrates how machine learning can be effectively used to support disaster preparedness and early flood warning systems. By analysing historical weather and rainfall data, the system successfully predicts flood occurrences using classification algorithms such as Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost. The best-performing model, XGBoost, achieved an accuracy of 96.55% and was integrated into a Flask-based web application to provide real-time flood risk predictions.

The project provided practical experience in data analysis, preprocessing, machine learning model development, model evaluation, and web application deployment. The intuitive user interface enables authorities and disaster management teams to quickly assess flood risks and make informed decisions. In the future, the system can be enhanced by integrating real-time weather APIs, automated alert notifications, and cloud-based deployment on IBM Cloud, making it a more comprehensive and scalable solution for flood disaster management.